NOTE: Selecting Question 3,4,5 for my exam

Question-3

Write down the forward pass and backward pass equations for a three layer convolution neural network (CNN). You are free to pick the design parameters (i.e., input/output dimension, number of kernels, their size, dilation, etc.) of your choice. You should provide the list of all design parameters for a CNN in general and the design parameters for your network. Define all the parameters/notations and their dimensions.

Written down the formulations and equations on paper and attached the images below.

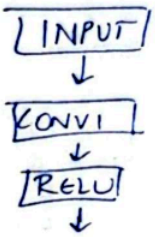
Assumed that since the equations were asked, this wasn't about code (but used this formulation in code for answering Q5)

CS584 - Mid Term RITVIK GARIMELLA

Three-layer CNN:

Input : Batch size (N) ; input channels (C_{in}) ; Spatial size ($H_0 \times W_0$)

for each convolutional Layer:
- output channels (C_L)
- Kernel size $K_{L,h} \times K_{L,w}$
- stride $S_{L,h}$, $S_{L,w}$
- padding $P_{L,h}$, $P_{L,w}$
- dilation $d_{L,h}$, $d_{L,w}$



for our CNN example -

Input : $X \in \mathbb{R}^{N \times C_0 \times H_0 \times W_0}$

Layer-1 : (Conv1 + Relu)

Out channel - C_1 ; Kernel $K_{1,h} \times K_{1,w}$; stride $(S_{1,h}, S_{1,w})$

Padding $(P_{1,h}, P_{1,w})$; dilation $(d_{1,h}, d_{1,w})$

Weights : $W^{(1)} \in \mathbb{R}^{C_1 \times C_0 \times K_{1,h} \times K_{1,w}}$, bias $b^{(1)} \in \mathbb{R}^{C_1}$

Output size :

$$H_1 = \left\lfloor \frac{H_0 + 2P_{1,h} - d_{1,h}(K_{1,h} - 1) - 1}{S_{1,h}} + 1 \right\rfloor$$

$$W_1 = \left\lfloor \frac{W_0 + 2P_{1,w} - d_{1,w}(K_{1,w} - 1) - 1}{S_{1,w}} + 1 \right\rfloor$$

Activations

$$A_1 \in \mathbb{R}^{N \times C_1 \times H_1 \times W_1}$$

Layer-2 (Conv2 + ReLU)

Out channel - C_2 ; Kernel : $K_{2,h} \times K_{2w}$; Stride - S_2
Padding - P_2
dilation - d_2

$W_2 \in \mathbb{R}^{C_2 \times C_1 \times K_{2,h} \times K_{2,w}}$, $b_2 \in \mathbb{R}^{C_2}$

Outsize H_2, W_2 computed as same as layer ①

$$H_2 = \left\lfloor \frac{H_1 + 2p_{2,h} - d_{2,h}(K_{2,h} - 1) - 1}{S_{2,h}} + 1 \right\rfloor$$

$$W_2 = \left\lfloor \frac{W_1 + 2p_{2,w} - d_{2,w}(K_{2,w} - 1) - 1}{S_{2,w}} + 1 \right\rfloor$$

$A_2 \in \mathbb{R}^{N \times C_2 \times H_2 \times W_2}$

Layer3 (Conv3 + ReLU)

Out - C_3 ; k : $K_{3,h} \times K_{3w}$; S_3, P_3, d_3 ; group: $g_3 = 1$

$W_3 \in \mathbb{R}^{C_3 \times C_2 \times K_{3,h} \times K_{3,w}}$ $b_3 \in \mathbb{R}^{C_3}$

Out size H_3, W_3 ; activations $A_3 \in \mathbb{R}^{N \times C_3 \times H_3 \times W_3}$

Head : Global Avg. pooling + Linear + Softmax

GAP : $G \in \mathbb{R}^{N \times C_3}$ $G_{N,C} = \frac{1}{H_3 W_3} \sum_{h,w} A^{(3)}_{n,c,h,w}$

Loss : mean cross-entropy

$$L = -\frac{1}{N} \sum_{n=1}^{N} \sum_{m=1}^{M} y_{n,m} \log P_{n,m}$$

Forward Pass

③

① Conv 1 + ReLU

$$Z_1 = \text{Conv}(X; W_1, b_1, s_1, p_1, d_1, g_1); \quad A_1 = \sigma(Z_1) \quad | \quad \sigma = \text{ReLU}$$

② Conv 2 + ReLU

$$Z_2 = \text{Conv}(A_1; W_2, b_2, s_2, p_2, d_2, g_2); \quad A_2 = \sigma(Z_2)$$

③ Conv 3 + ReLU

$$Z_3 = \text{Conv}(A_2; W_3, b_3, s_3, p_3, d_3, g_3); \quad A_3 = \sigma(Z_3)$$

④ GAP + Classifier + Softmax

$$G_{n,c} = \frac{1}{H_3 W_3} \sum_{h,w} A^{(3)}_{n,c,h,w} \qquad S = GU + 1c^T \qquad P_{n,m} = \frac{e^{S_{n,m}}}{\sum_{j=1}^{M} e^{S_{n,j}}}$$

Backward pass

$$\delta_L = \frac{\partial L}{\partial Z_L} \qquad \bar{T} = \frac{\partial L}{\partial T}$$

0) Softmax + cross-entropy

$$\bar{S} = \frac{\partial L}{\partial S} = \frac{1}{N}(P - Y) \in \mathbb{R}^{N \times M}$$

1) Linear classifier

$$\bar{U} = G^T \bar{S} \in \mathbb{R}^{C_3 \times M}, \quad c = \sum_{n=1}^{N} \bar{S}_n \in \mathbb{R}^M, \quad \bar{G} = \bar{S} U^T \in \mathbb{R}^{N \times C_3}$$

2) Back through GAP

$$\bar{A}^{(3)}_{n,c,h,w} = \frac{1}{H_3 W_3} \bar{G}_{n,c} \in \mathbb{R}^{N \times C_3 \times H_3 \times W_3}$$

3) ReLU of layer 3 $\quad \delta^{(3)} = \bar{A}^{(3)} \odot 1\{Z^{(3)} > 0\}$

④

4) Conv3 parameter & input gradients

- Bias $\bar{b}_c^{(3)} = \sum_{n,h,w} \delta^{(3)}_{n,c,h,w} \in \mathbb{R}^{C_3}$

- Weights $\bar{W}^{(3)}_{C_{out}, C_{in}, i, j} = \sum_{n,h,w} \delta^{(3)}_{n, C_{out}, h, w} \cdot A^{(2)}_{n, C_{in}, h', w'}$

(h', w') – input coordinates sampled by forward conv. (h, w, i, j) under (s_3, p_3, d_3)

$$\bar{W}^{(3)} = \sum_n \delta_n^{(3)} * A_n^{(2)}$$

Input (to feed to previous layer)

$$\bar{A}^{(2)} = \text{ConvTranspose}(\delta^{(3)}; W^{(3)}, s_3, p_3, d_3, g_3)$$

5) ReLU of layer 2 : $\delta^{(2)} = \bar{A}^{(2)} \odot 1\{Z^{(2)} > 0\}$

6) Conv2 gradients (same pattern)

$$\bar{b}_2^{(2)} = \sum_{n,h,w} \delta^{(2)}_{n,c,h,w} \qquad \bar{W}^{(2)} = \sum_n \delta_n^{(2)} * A_n^{(1)}$$

$$\bar{A}^{(1)} = \text{ConvTranspose}(\delta^{(2)}; W^{(2)}, s_2, p_2, d_2, g_2)$$

7) $\delta^{(1)} = \bar{A}^{(1)} \odot 1(Z^{(1)} > 0)$

8) Conv1 gradients

$$\bar{b}_c^{(1)} = \sum_{n,h,w} \delta^{(1)}_{n,c,h,w} \qquad \bar{W}^{(1)} = \sum_n \delta_n^{(1)} * X_n$$

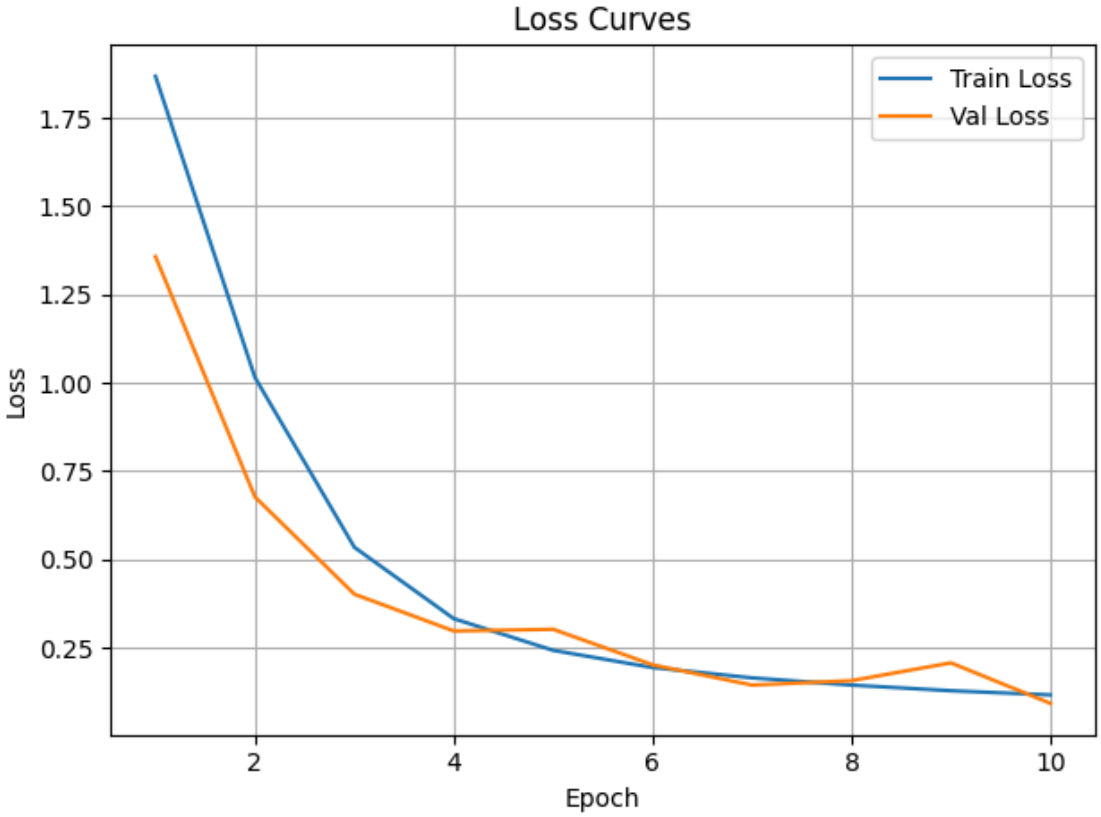
$$\bar{x} = \text{ConvTranspose}(\delta_{(1)}; W^{(1)}, s_1, p_1, d_1, g_1)$$

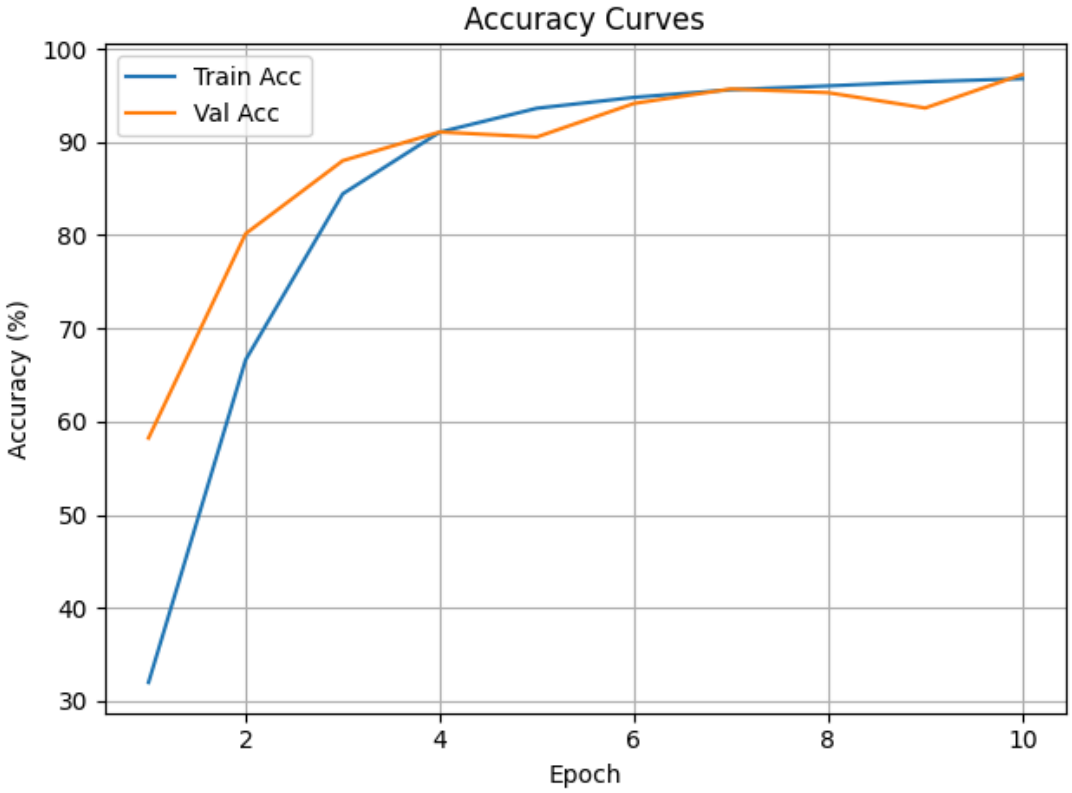
Question-4

For the MNIST dataset used in your HW 1 (see link to article below), write down a custom code to train a CNN with three convolutional layers for the handwritten digit classification problem. Article: Classification of handwritten digits – this is the same code you have submitted for a HW 1 problem.

Link to code - https://github.com/Ritvik-G/USC_CSCE584/blob/main/mid_term/codes/Q4.py

```
(base) ritvikg@Ritviks-MacBook-Pro mid term % python Q4.py
Device: cpu
/Users/ritvikg/Desktop/USC/Classes/CS584/mid term/Q4.py:170: FutureWarning: `torch.cuda.amp.GradScaler(args...)`
  scaler = torch.cuda.amp.GradScaler(enabled=use_amp)
Epoch 01 | train loss 1.8683 acc 32.00% | val loss 1.3576 acc 58.23%
Epoch 02 | train loss 1.0153 acc 66.62% | val loss 0.6768 acc 80.16%
Epoch 03 | train loss 0.5342 acc 84.43% | val loss 0.4013 acc 87.99%
Epoch 04 | train loss 0.3319 acc 91.06% | val loss 0.2970 acc 91.07%
Epoch 05 | train loss 0.2421 acc 93.62% | val loss 0.3018 acc 90.53%
Epoch 06 | train loss 0.1934 acc 94.78% | val loss 0.2010 acc 94.13%
Epoch 07 | train loss 0.1641 acc 95.60% | val loss 0.1438 acc 95.69%
Epoch 08 | train loss 0.1437 acc 96.02% | val loss 0.1562 acc 95.29%
Epoch 09 | train loss 0.1277 acc 96.46% | val loss 0.2067 acc 93.64%
Epoch 10 | train loss 0.1159 acc 96.79% | val loss 0.0920 acc 97.24%
Test  loss 0.0870 acc 97.49%
Best val acc: 97.24% | Model saved to: /Users/ritvikg/Desktop/USC/Classes/CS584/mid term/mnist_cnn_manual.pt
Saved plots to: /Users/ritvikg/Desktop/USC/Classes/CS584/mid term/curves.png and /Users/ritvikg/Desktop/USC/Class
```


Loss Curves
Train Loss
Val Loss
Loss
1.75
1.50
1.25
1.00
0.75
0.50
0.25
2
4
6
8
10
Epoch

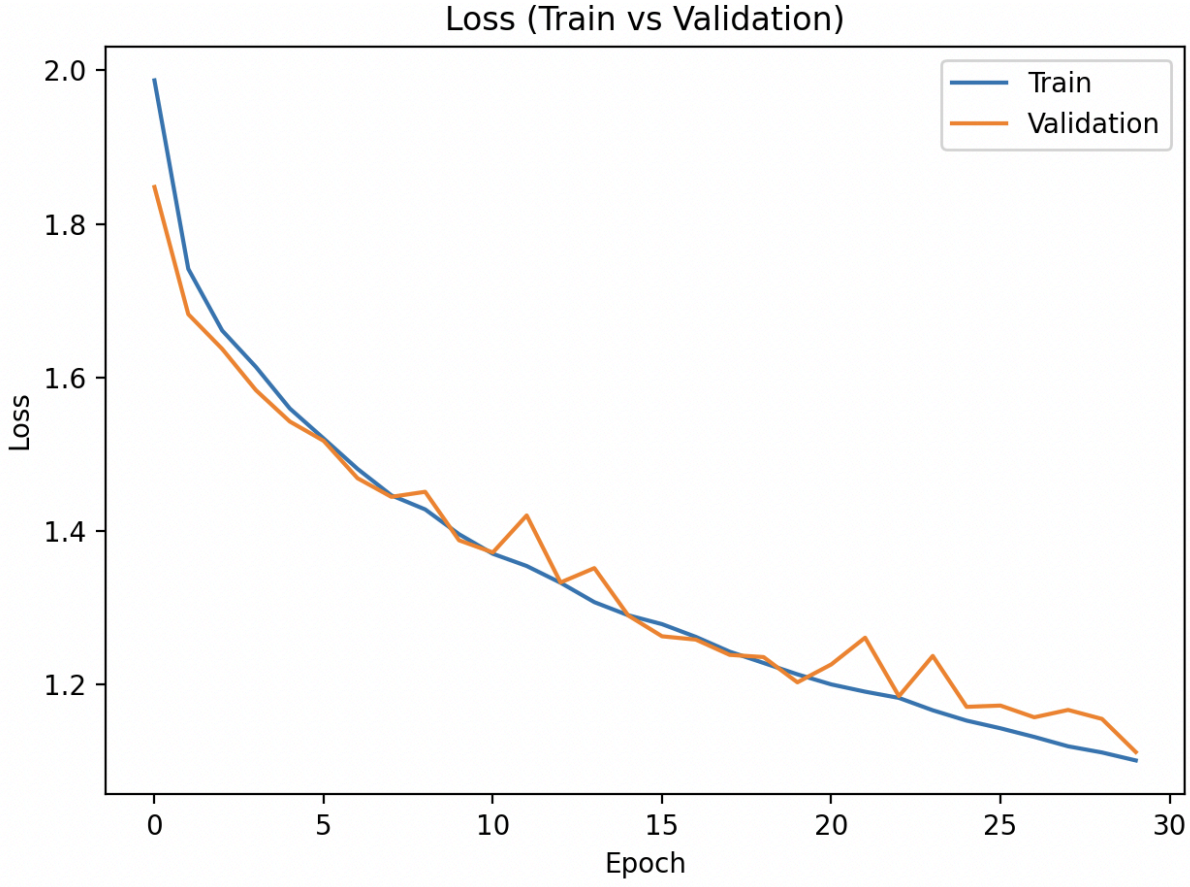

Accuracy Curves
Train Acc
Val Acc
Accuracy (%)
100
90
80
70
60
50
40
30
2
4
6
8
10
Epoch

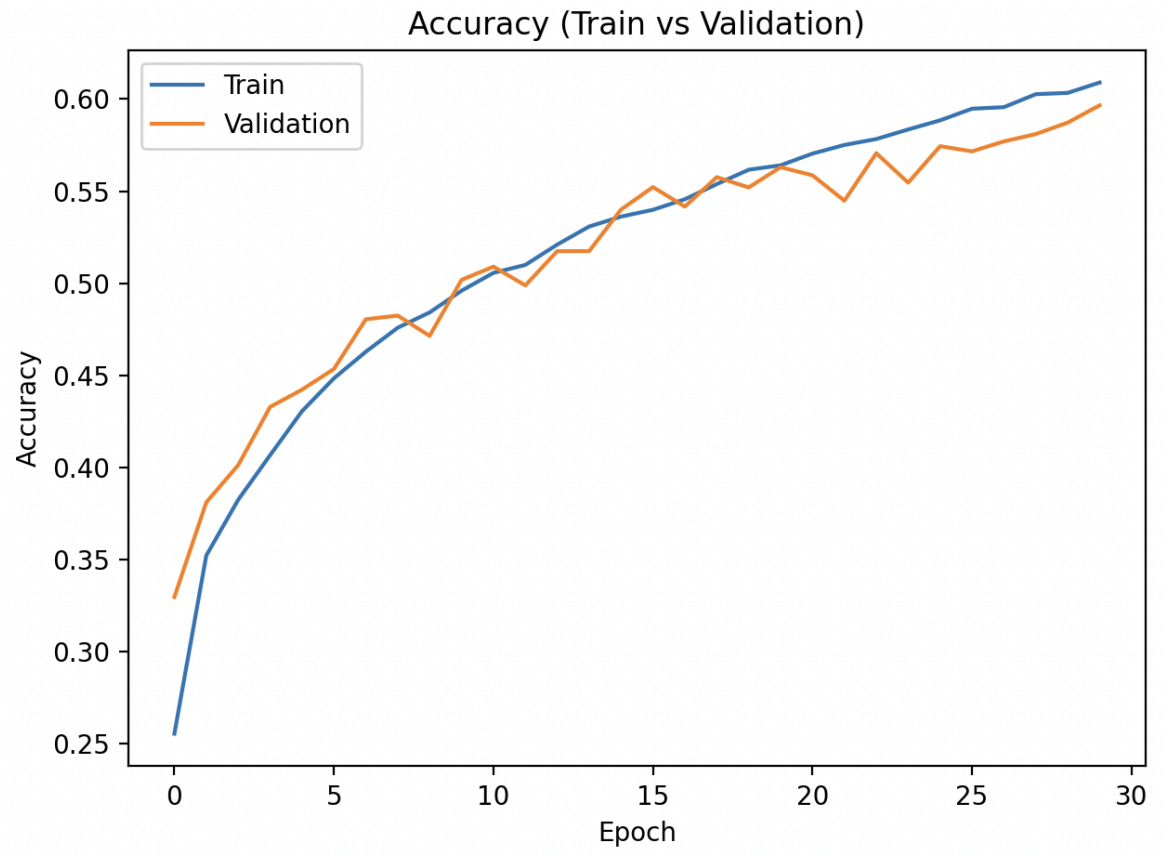
Question-5

Use the code for digit classification in HW1 from the article (or your solution to Problem 3 of this exam): Code for classification of handwritten digits. Instead of using the MNIST dataset, use the CIFAR-10 dataset (to perform 10 class classification). Download/Use the dataset from the link given here: CIFAR dataset.

Link to code - https://github.com/Ritvik-G/USC_CSCE584/blob/main/mid_term/codes/Q5.py

```
(base) ritvikg@Ritviks-MacBook-Pro mid term % python Q5.py
Epoch 01: Train loss=1.987, acc=25.54% | Val loss=1.848, acc=32.96%
Epoch 02: Train loss=1.741, acc=35.21% | Val loss=1.682, acc=38.10%
Epoch 03: Train loss=1.661, acc=38.24% | Val loss=1.637, acc=40.12%
Epoch 04: Train loss=1.614, acc=40.66% | Val loss=1.583, acc=43.28%
Epoch 05: Train loss=1.560, acc=43.05% | Val loss=1.543, acc=44.22%
Epoch 06: Train loss=1.520, acc=44.83% | Val loss=1.517, acc=45.34%
Epoch 07: Train loss=1.481, acc=46.28% | Val loss=1.469, acc=48.04%
Epoch 08: Train loss=1.447, acc=47.58% | Val loss=1.445, acc=48.24%
Epoch 09: Train loss=1.428, acc=48.41% | Val loss=1.451, acc=47.14%
Epoch 10: Train loss=1.396, acc=49.59% | Val loss=1.388, acc=50.18%
Epoch 11: Train loss=1.370, acc=50.56% | Val loss=1.372, acc=50.90%
Epoch 12: Train loss=1.355, acc=50.99% | Val loss=1.420, acc=49.88%
Epoch 13: Train loss=1.333, acc=52.10% | Val loss=1.333, acc=51.74%
Epoch 14: Train loss=1.307, acc=53.08% | Val loss=1.352, acc=51.74%
Epoch 15: Train loss=1.290, acc=53.62% | Val loss=1.290, acc=54.00%
Epoch 16: Train loss=1.279, acc=53.99% | Val loss=1.263, acc=55.22%
Epoch 17: Train loss=1.262, acc=54.56% | Val loss=1.259, acc=54.16%
Epoch 18: Train loss=1.243, acc=55.38% | Val loss=1.239, acc=55.76%
Epoch 19: Train loss=1.228, acc=56.16% | Val loss=1.236, acc=55.20%
Epoch 20: Train loss=1.213, acc=56.40% | Val loss=1.203, acc=56.30%
Epoch 21: Train loss=1.200, acc=57.04% | Val loss=1.226, acc=55.86%
Epoch 22: Train loss=1.191, acc=57.51% | Val loss=1.261, acc=54.48%
Epoch 23: Train loss=1.183, acc=57.83% | Val loss=1.185, acc=57.06%
Epoch 24: Train loss=1.167, acc=58.35% | Val loss=1.237, acc=55.46%
Epoch 25: Train loss=1.153, acc=58.84% | Val loss=1.171, acc=57.44%
Epoch 26: Train loss=1.143, acc=59.48% | Val loss=1.173, acc=57.16%
Epoch 27: Train loss=1.132, acc=59.56% | Val loss=1.158, acc=57.70%
Epoch 28: Train loss=1.120, acc=60.26% | Val loss=1.167, acc=58.10%
Epoch 29: Train loss=1.112, acc=60.34% | Val loss=1.155, acc=58.72%
Epoch 30: Train loss=1.101, acc=60.90% | Val loss=1.112, acc=59.66%
Saved plots to figs/loss_curve.png and figs/accuracy_curve.png
Loaded best model (Val acc = 59.66%).
Final Test: loss=1.111, acc=60.61%
Model saved as 'three_layer_cnn_cifar10.pth'.
```

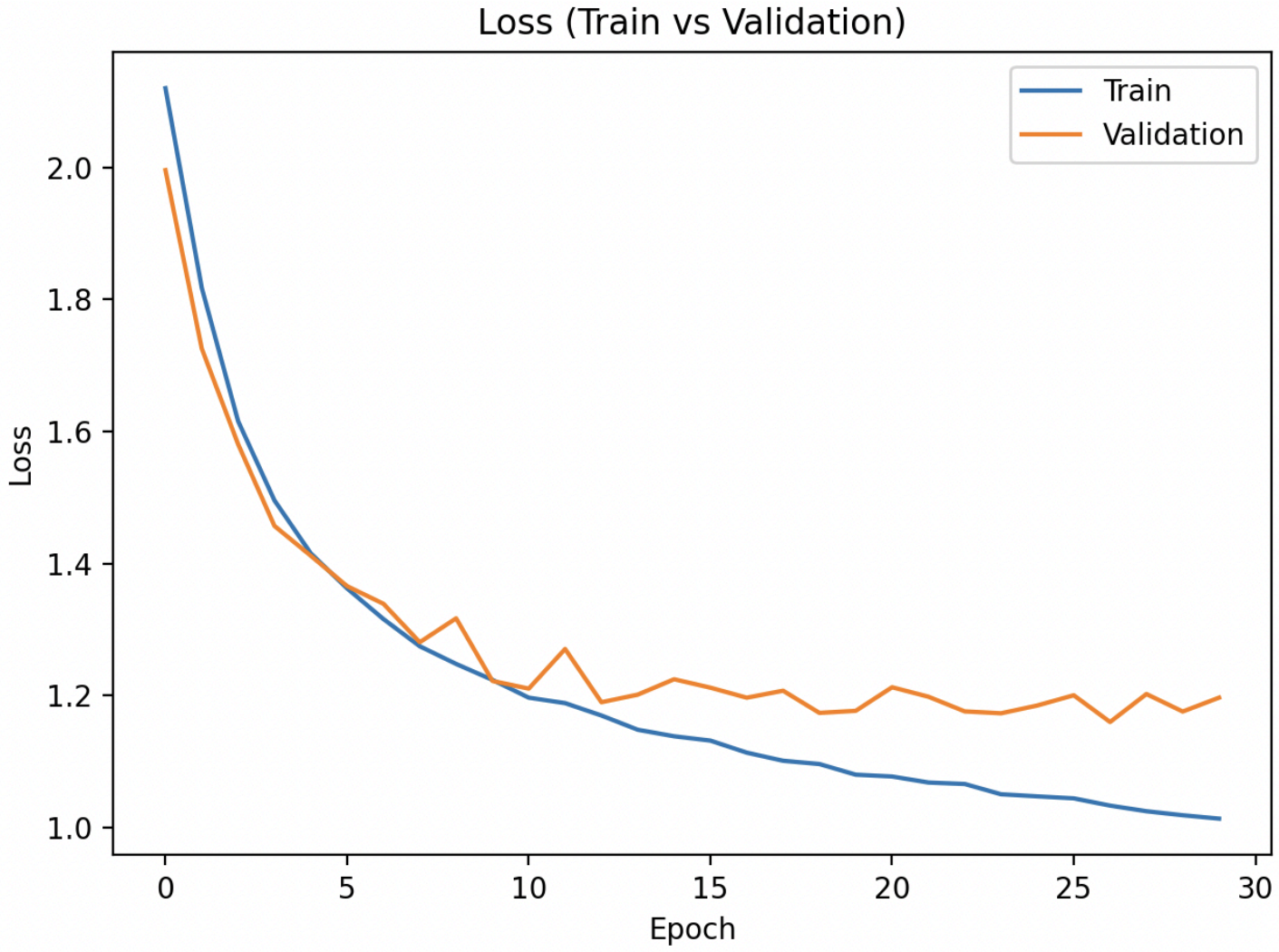

Loss (Train vs Validation)
Train
Validation
Loss
Epoch
2.0
1.8
1.6
1.4
1.2
0
5
10
15
20
25
30

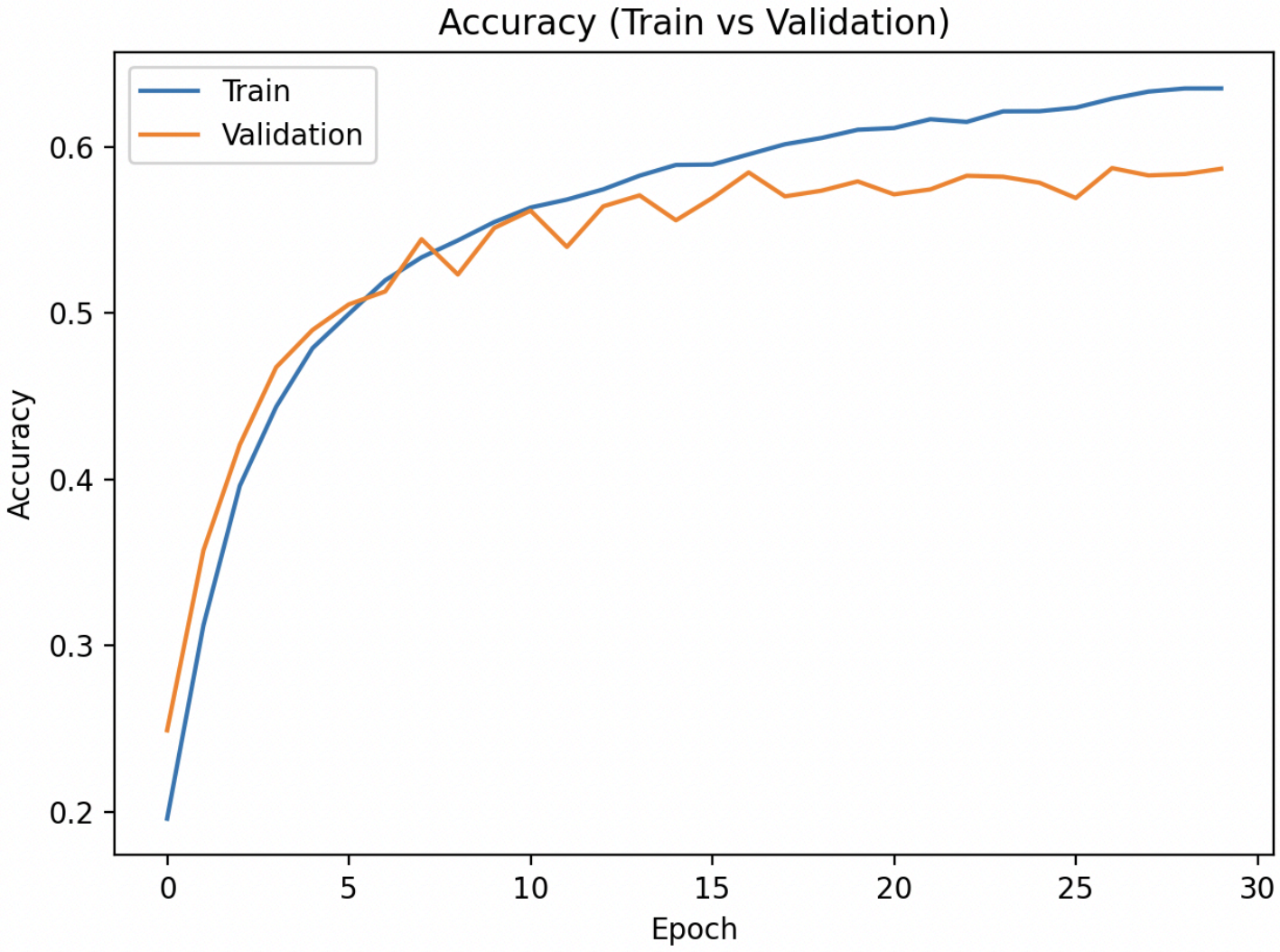

Accuracy (Train vs Validation)
Train
Validation
Accuracy
Epoch
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0
5
10
15
20
25
30

NOTE: I've also tried adding fully-connected hidden layers to see if there would be any change in the results and bumped up the learning rate from 0.001 to 0.01. Below are the results and respective code -

Code: https://github.com/Ritvik-G/USC_CSCE584/blob/main/mid_term/codes/Q5_modified.py

```
(base) ritvikg@Ritviks-MacBook-Pro codes % python Q5_modified.py
Epoch 01: Train loss=2.120, acc=19.58% | Val loss=1.996, acc=24.90%
Epoch 02: Train loss=1.818, acc=31.22% | Val loss=1.726, acc=35.72%
Epoch 03: Train loss=1.616, acc=39.61% | Val loss=1.581, acc=42.08%
Epoch 04: Train loss=1.496, acc=44.36% | Val loss=1.457, acc=46.74%
Epoch 05: Train loss=1.415, acc=47.89% | Val loss=1.412, acc=48.98%
Epoch 06: Train loss=1.362, acc=49.95% | Val loss=1.366, acc=50.52%
Epoch 07: Train loss=1.316, acc=51.97% | Val loss=1.339, acc=51.30%
Epoch 08: Train loss=1.275, acc=53.35% | Val loss=1.281, acc=54.44%
Epoch 09: Train loss=1.248, acc=54.37% | Val loss=1.317, acc=52.32%
Epoch 10: Train loss=1.224, acc=55.47% | Val loss=1.222, acc=55.12%
Epoch 11: Train loss=1.197, acc=56.35% | Val loss=1.211, acc=56.16%
Epoch 12: Train loss=1.189, acc=56.83% | Val loss=1.271, acc=53.98%
Epoch 13: Train loss=1.170, acc=57.44% | Val loss=1.190, acc=56.42%
Epoch 14: Train loss=1.148, acc=58.26% | Val loss=1.201, acc=57.08%
Epoch 15: Train loss=1.138, acc=58.91% | Val loss=1.225, acc=55.58%
Epoch 16: Train loss=1.132, acc=58.93% | Val loss=1.212, acc=56.92%
Epoch 17: Train loss=1.114, acc=59.55% | Val loss=1.197, acc=58.46%
Epoch 18: Train loss=1.101, acc=60.16% | Val loss=1.207, acc=57.02%
Epoch 19: Train loss=1.096, acc=60.53% | Val loss=1.174, acc=57.36%
Epoch 20: Train loss=1.080, acc=61.03% | Val loss=1.177, acc=57.92%
Epoch 21: Train loss=1.077, acc=61.13% | Val loss=1.213, acc=57.14%
Epoch 22: Train loss=1.068, acc=61.66% | Val loss=1.198, acc=57.44%
Epoch 23: Train loss=1.066, acc=61.50% | Val loss=1.176, acc=58.26%
Epoch 24: Train loss=1.051, acc=62.13% | Val loss=1.173, acc=58.20%
Epoch 25: Train loss=1.047, acc=62.14% | Val loss=1.185, acc=57.84%
Epoch 26: Train loss=1.044, acc=62.36% | Val loss=1.201, acc=56.92%
Epoch 27: Train loss=1.033, acc=62.90% | Val loss=1.160, acc=58.72%
Epoch 28: Train loss=1.025, acc=63.32% | Val loss=1.202, acc=58.28%
Epoch 29: Train loss=1.019, acc=63.51% | Val loss=1.176, acc=58.36%
Epoch 30: Train loss=1.014, acc=63.51% | Val loss=1.197, acc=58.68%
Saved plots to figs/loss_curve.png and figs/accuracy_curve.png
Loaded best model (Val acc = 58.72%).
Final Test: loss=1.186, acc=58.32%
Model saved as 'three_layer_cnn_cifar10.pth'.
(base) ritvikg@Ritviks-MacBook-Pro codes %
```


Loss (Train vs Validation)
Train
Validation
Loss
2.0
1.8
1.6
1.4
1.2
1.0
0
5
10
15
20
25
30
Epoch


Accuracy (Train vs Validation)
Train
Validation
Accuracy
0.2
0.3
0.4
0.5
0.6
0
5
10
15
20
25
30
Epoch